

最简单vpn (tap/tun) --rust



putao

2024-08-26 北京 阅读 7 分钟

提前准备：

TAP (Network TAP)：工作在数据链路层（第2层）。

TUN (Network TUNnel)：工作在网络层（第3层）。传输加密的 IP 数据包，从而实现点到点的加密通信。虚拟的物理网络接口，用户态可以直接写数据。数据包可以被操作系统协议栈进一步处理，就像该数据包是通过一个物理网络接口接收到的一样。

TAP设备：

数据链路层设备：模拟以太网设备，处理以太网帧。

无需网络协议栈处理：数据帧直接传递，不需要进入内核的网络协议栈。

内核传输这些帧：

这些数据帧可以直接被其它网络工具处理或发送到实际物理接口。

内核中的 bridge 设备将这些帧传递到物理接口，或者转发给其他虚拟设备。

TUN设备：

网络层设备：模拟网络层设备，处理IP包。

需要网络协议栈处理：IP包需要进入内核网络协议栈进行处理。

内核处理这些包：

内核接收这些IP包进行网络层处理（如路由决策、转发、NAT等）。

内核将处理后的IP包发送到实际物理接口，或交给其他网络程序进行处理。

如何使用tap呢？

```
// TAP 配置
let mut config = Configuration::default();
config
    .address((192, 168, 100, 1))
```

```
.netmask((255, 255, 255, 0))
.tap(true) // 指定使用 TAP 模式
.up();

// 创建并配置 TAP 设备
let mut dev = Device::new(&config).unwrap();
println!("TAP device created");
```

杂问杂答：A和B想要通过vpn进行通信。

- 网络协议：可以有很多种方式，这里使用的udp.
- udp里面传的是什麼数据呢？ wireguard协议的数据， 一种vpn 协议。简单版本先不用考虑这个， 用一个加密的数据即可。
- 整个过程有没有内核参与呢？ 有！！！！
- 如果是tun的方式：用户态的！
- 用户态加解密数据， 写入到tun(虚拟的网卡)
- 数据是怎么走到虚拟网卡的？ 需要配置路由协议。
- 分配给tun的IP是虚拟的， 随便设计IP的地址吧， 但是要有， 有了才能走路由规则。

服务依赖：

```
rand = "0.8.5"
tun = "0.6.1"
x25519-dalek = { version = "2.0.1", features = ["static_scalar"] }
blake2 = "0.11.0-pre.4"
```

服务端：

```

fn decrypt_data(server_privkey: &StaticSecret, client_public_key: &PublicKey) {
    let shared_secret = server_privkey.diffie_hellman(client_public_key);
    let key = blake2_key_derive(&shared_secret);
    // 使用对称加密（例如 ChaCha20）的逻辑进行解密
    data.to_vec() // 暂时返回原数据
}

fn blake2_key_derive(shared_secret: &x25519_dalek::SharedSecret) {
    let mut hasher = Blake2s::new_keyed(shared_secret.as_bytes());
    hasher.update(b"WireGuard");
    let mut key = vec![0u8; 32];
    hasher.finalize_variable(|res| key.copy_from_slice(res));
    key
}

```

客户端：

```

use tokio::{
    net::UdpSocket,
    io::{self, AsyncReadExt, AsyncWriteExt},
};
use tun::platform::Device;
use tun::Configuration;
use x25519_dalek::StaticSecret;
use x25519_dalek::PublicKey;
use blake2::Blake2s;
use blake2::digest::{Update, VariableOutput};
use rand::rngs::OsRng;
use std::process::Command;

pub async fn run_client() {
    // TUN 配置
    let mut config = Configuration::default();
    config.address((10, 0, 0, 2)).netmask((255, 255, 255, 0));

    let mut dev = Device::new(&config).unwrap();
    println!("Client TUN device created");

    // 配置路由
    configure_routing();

    // 加密密钥
    let client_privkey = StaticSecret::new(&mut OsRng);
}

```

👍 赞 1

🔖 收藏 1

🔗 分享

阅读 1.2k · 更新于 2024-08-27



putao

8 声望 · 3 粉丝 · 🌐 📺

推动世界向前发展，改善民生。

关注作者

« 上一篇

内网穿透技术-- rust

下一篇 »

vpn里面的加密处理--rust

引用和评论

推荐阅读



flink的窗口计算方式

putao · 阅读 347



vite5+tauri2.0客户端仿微信Exe聊天程序|tauri2+rust+vue3聊天实例

xiaoyan2017 · 阅读 10.7k



rust 命令行工具rsup管理前端npm依赖

hboot · 赞 1 · 阅读 1.4k



支付宝H5下载被拦截的原因排查与解决指南

云轻雨细 · 阅读 1.7k



放弃 Rust 选择 Zig，Xata 团队推出 pgzx —— 计划使用 Zig 开发基于 PG 的分布式数据库

小猿姐 · 阅读 1.4k



发送http请求

putao · 阅读 901



RUST练习生如何在生产环境构建万亿流量|得物技术

得物技术 · 阅读 860

1 条评论

得票

最新



撰写评论 ...



提交评论

评论支持部分 Markdown 语法: ****粗体**** *_斜体_* [\[链接\]](#)
(<http://example.com>) ``代码`` - 列表 > 引用。你还可以使用
[@](#) 来通知其他用户。



没有腹肌的芹菜: 博主你好, 有个问题请教一下, 按照这个c/s程序, 客户端将要发的数据通过socket发给服务器之后, 服务器解密解包之后写入tun, 再从tun读响应的数据吗? 这个程序应该能实现客户端ping服务器的虚拟网卡的ip, 但是如果客户端ping一个公网地址 (如8.8.8.8), 这个程序是不是不能满足这个要求呢? 如果我要实现这个功能, 服务器端是需要将从客户端收到的数据用socket连接8.8.8.8然后转发出去并等待响应吗? 还是应该是通过tun或者其他什么方法实现吗

👍 · 回复 · 2024-10-12 · 来自陕西

©2025 putao0525

除特别声明外, 作品采用《署名-非商业性使用-禁止演绎 4.0 国际》进行许可

 使用 SegmentFault 发布

SegmentFault - 凝聚集体智慧, 推动技术进步

服务协议 · 隐私政策 · 浙ICP备15005796号-2 · 浙公网安备33010602002000号